

JavaScript

**: Consumindo
e tratando
dados de uma
API**



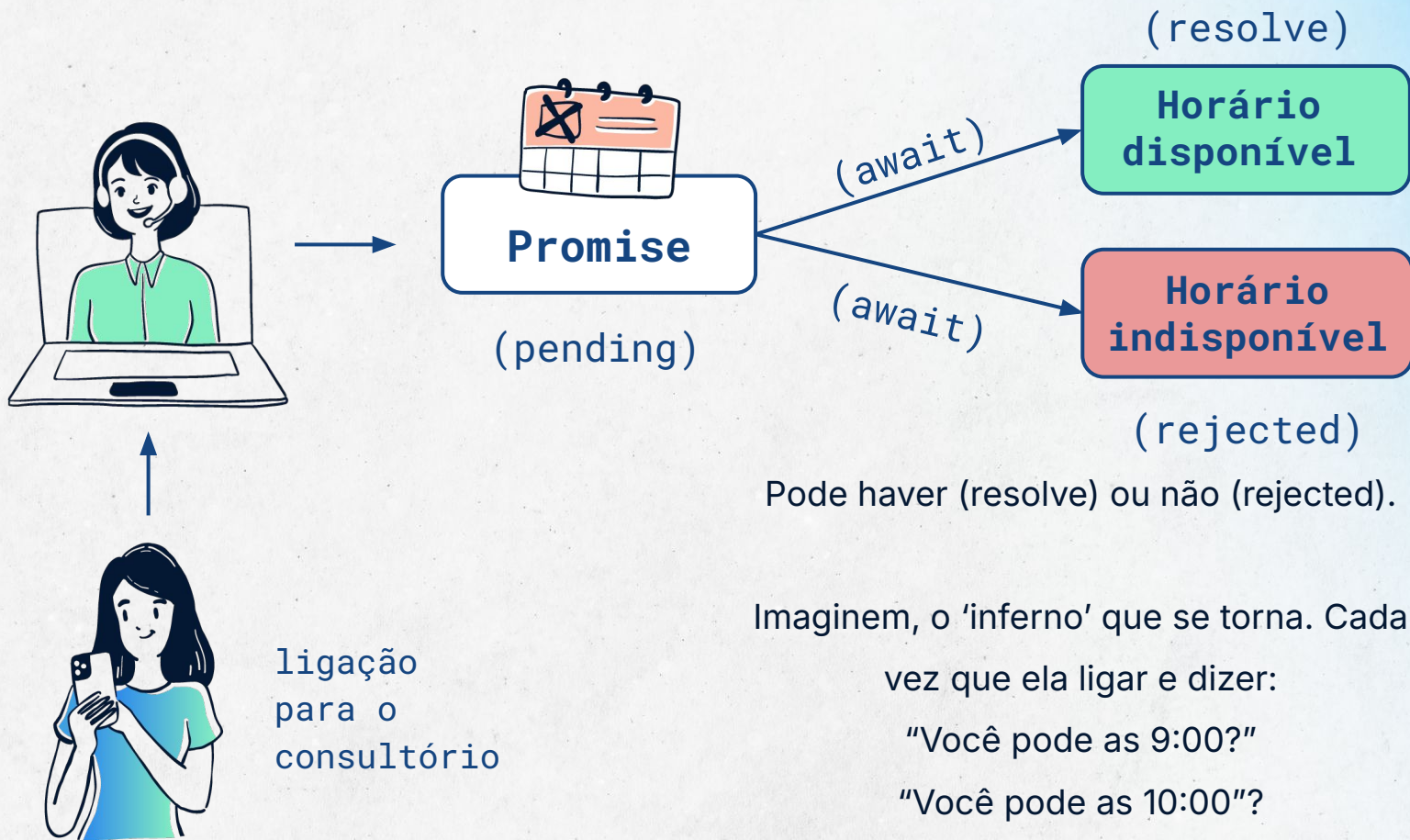


ligação
para o
consultório

Vamos pensar no uso do `async await` em
uma situação do cotidiano?
Você precisa consultar com um dentista.



Você telefona para o consultório para tentar uma consulta na terça-feira, e a secretária (gerando uma promise) pede pra aguardar que ela consultará a disponibilidade de horários (await).



Pode haver (resolve) ou não (rejected).

Imaginem, o 'inferno' que se torna. Cada

vez que ela ligar e dizer:

"Você pode as 9:00?"

"Você pode as 10:00"?

...

Funções assíncrona s com async



JS script.js > ...

```
1  async function buscarEndereco() {  
2  
3      var consultaCep = await fetch('https://viacep.com.br/ws/35188000/json/');  
4      var consultaCepConvertida = await consultaCep.json();  
5      console.log(consultaCepConvertida);  
6  
7  }  
8  buscarEndereco();
```

Mais simples, mais rápido e mais objetivo!

JS script.js > ...

```
1  async function buscarEndereco() {
2
3      var consultaCep = await fetch('https://viacep.com.br/ws/35188000/json/');
4      var consultaCepConvertida = await consultaCep.json();
5      console.log(consultaCepConvertida);
6
7  }
8  buscarEndereco();
```

▼ {cep: '35188-000', Logradouro: '', complemento: '', unidade: '',
bairro: ''} [script.js:4](#)

```
{
  cep: "35188-000",
  complemento: "",
  ddd: "31",
  estado: "Minas Gerais",
  gia: "",
  ibge: "3135001",
  localidade: "Jaguaraçu",
  logradouro: "",
  regioao: "Sudeste",
  siafi: "4699",
  uf: "MG",
  unidade: ""
}
```

► [[Prototype]]: Object

▼ {erro: 'true'} [script.js:4](#)

```
erro: "true"
  ► [[Prototype]]: Object
```

✖ Access to fetch at 'https://viacep.com.br/ws/3518800/json/' from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. [index.html:1](#)

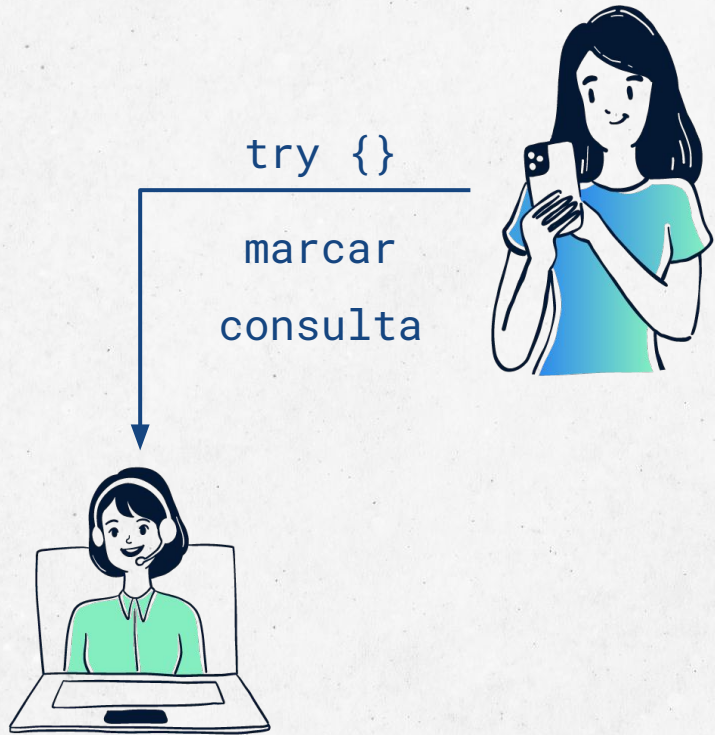
✖ ▶ GET https://viacep.com.br/ws/3518800/json/ net::ERR_FAILED 400 (Bad Request) [script.js:2](#)

[NOVO] Explique os erros do Console usando o Copilot no Edge: clique em [Saiba mais](#) para explicar um erro. [Não mostrar novamente](#)

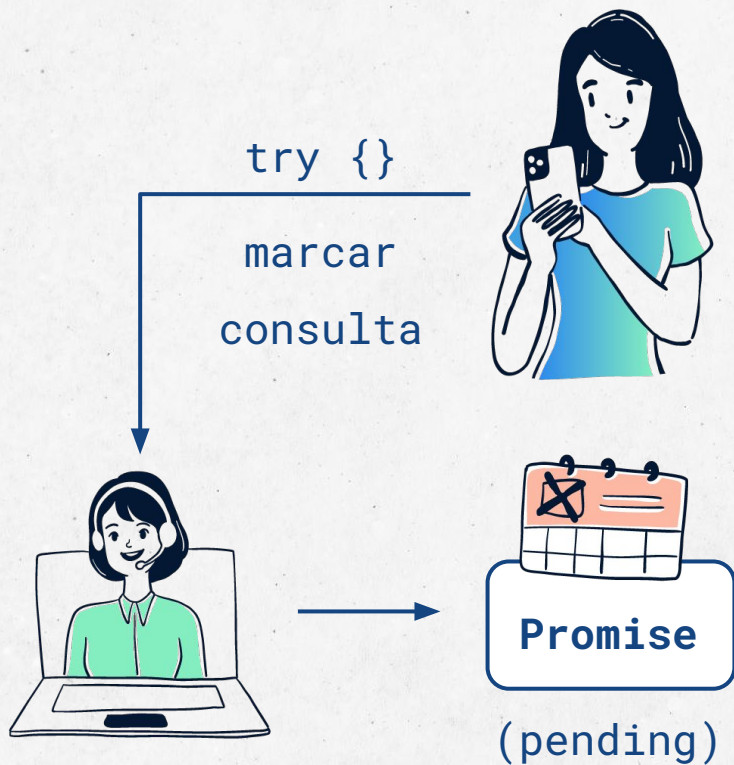
✖ ▶ Uncaught (in promise) TypeError: Failed to fetch at buscarEndereco (script.js:2:21) at script.js:7:1

Erros com async

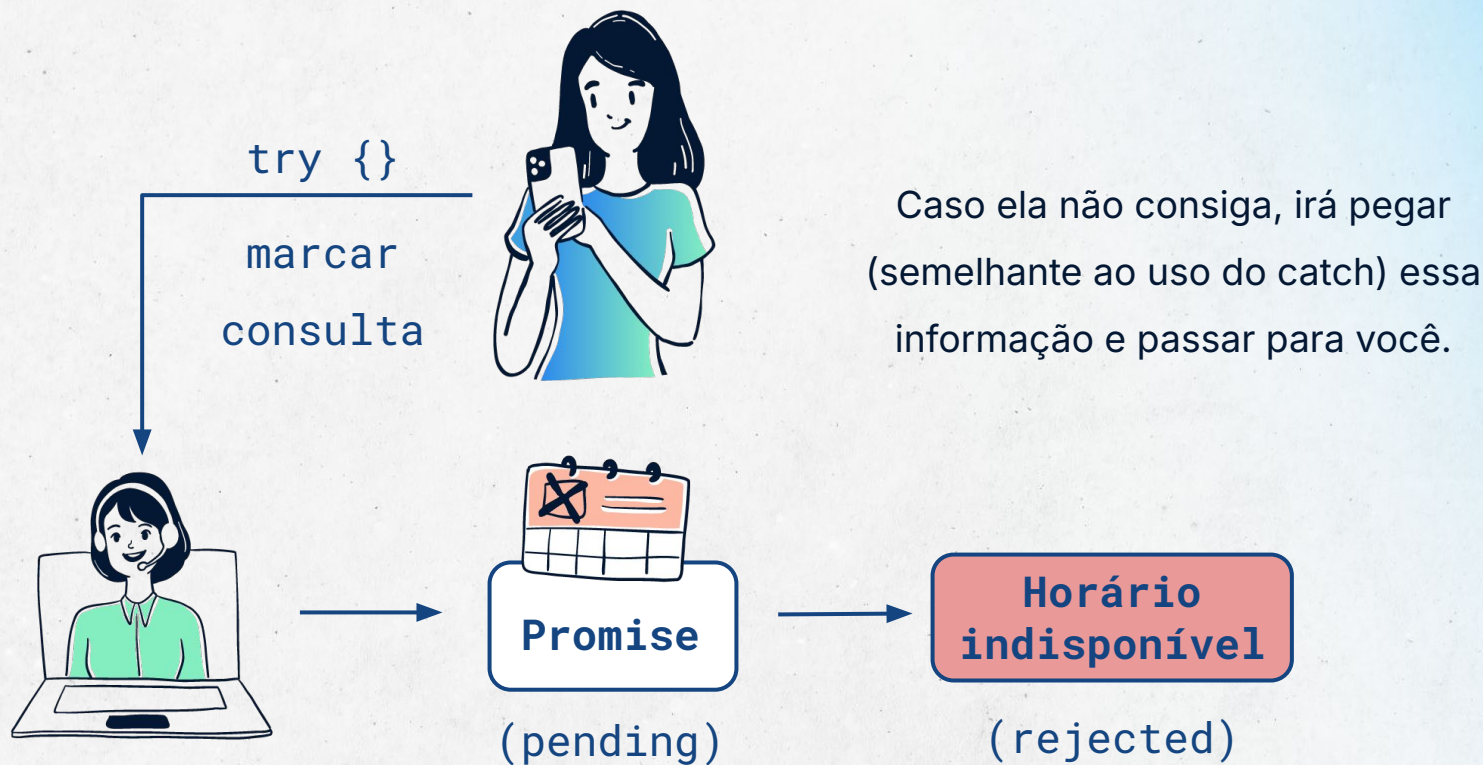


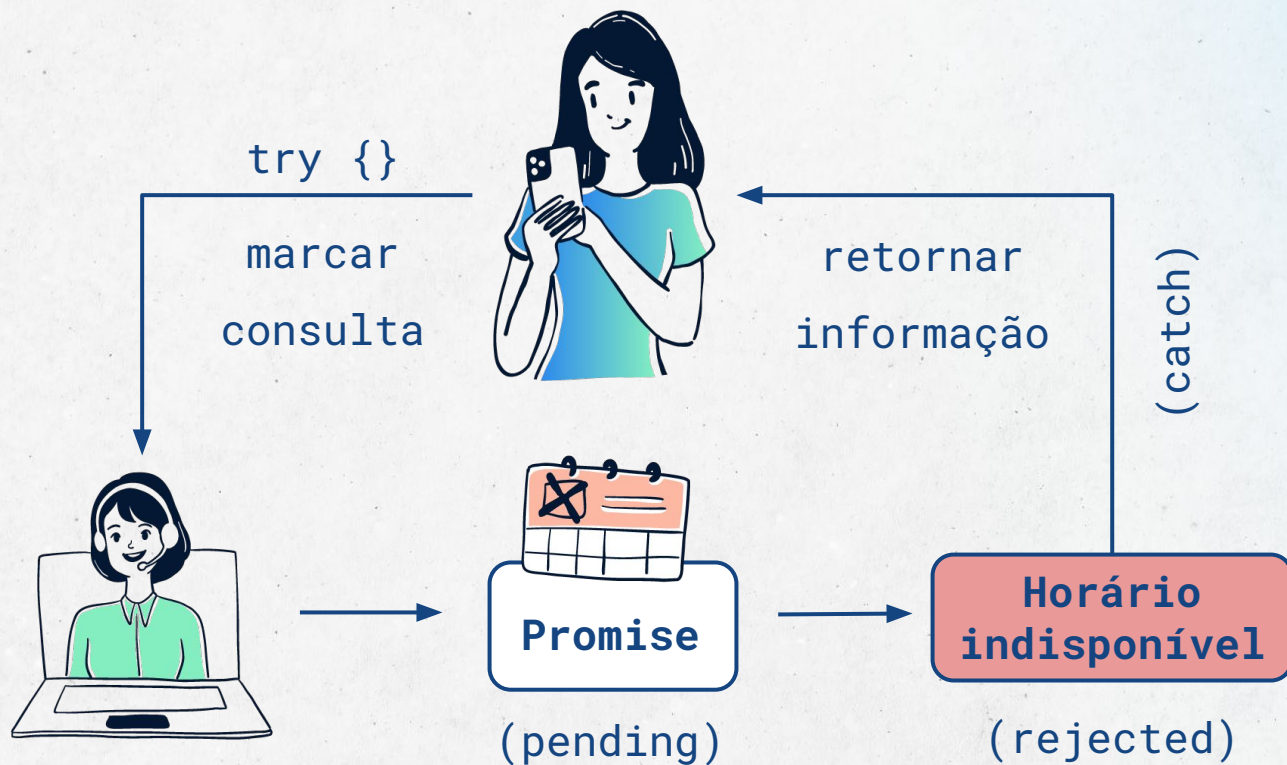


Lembra da consulta que você estava
tentando marcar?



Caso não seja possível marcar
naquele dia, a secretária irá tentar (o
try do tratamento de erros) agendar
uma consulta.





JS script.js > ...

```
1  async function buscarEndereco() {  
2      try {  
3          var consultaCep = await fetch('https://viacep.com.br/ws/35188000/json/');  
4          var consultaCepConvertida = await consultaCep.json();  
5          console.log(consultaCepConvertida);  
6      } catch (error) {  
7          console.log(error);  
8      }  
9  }  
10 buscarEndereco();
```


JS script.js > ...

```
1  async function buscarEndereco() {
2      try {
3          var consultaCep = await fetch('https://viacep.com.br/ws/35188000/json/');
4          var consultaCepConvertida = await consultaCep.json();
5          console.log(consultaCepConvertida);
6      } catch (error) {
7          console.log(error);
8      }
9  }
10 buscarEndereco();
```

✖ Access to fetch at '<https://viacep.com.br/ws/3518800/json/>' from origin '<http://127.0.0.1:5500>' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

[NOVO] Explique os erros do Console usando o Copilot no Edge: clique em ⓘ para explicar um erro. [Saiba mais](#)

Não mostrar novamente

✖ ▶ GET <https://viacep.com.br/ws/3518800/json/> script.js:3 ⓘ ⓘ
net::ERR_FAILED 400 (Bad Request)

TypeError: Failed to fetch script.js:7
at buscarEndereco (script.js:29)
at script.js:10:3

> Esse erro acontece se o CEP for incompleto

▼ {erro: 'true'} ⓘ script.js:5
erro: "true"
▶ [[Prototype]]: Object

>

Esse erro acontece se o CEP estiver errado

JS script.js > ...

```
1  async function buscarEndereco() {
2      try {
3          var consultaCep = await fetch('https://viacep.com.br/ws/35188000/json/');
4          var consultaCepConvertida = await consultaCep.json();
5          console.log(consultaCepConvertida);
6      } catch (error) {
7          console.log(error);
8      }
9  }
10 buscarEndereco();
```

✖ Access to fetch at 'https://viacep.com.br/ws/3518800/json/' from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. [index.html:1](#)

[NOVO] Explique os erros do Console usando o Copilot no Edge: clique em [?](#) para explicar um erro. [Saiba mais](#)

Não mostrar novamente

✖ ▶ GET https://viacep.com.br/ws/3518800/json/ script.js:3 [?](#)
net::ERR_FAILED 400 (Bad Request)

TypeError: Failed to fetch [script.js:7](#)
at buscarEndereco ([script.js:29](#))
at [script.js:10:3](#)

> Esse erro acontece se o CEP for incompleto

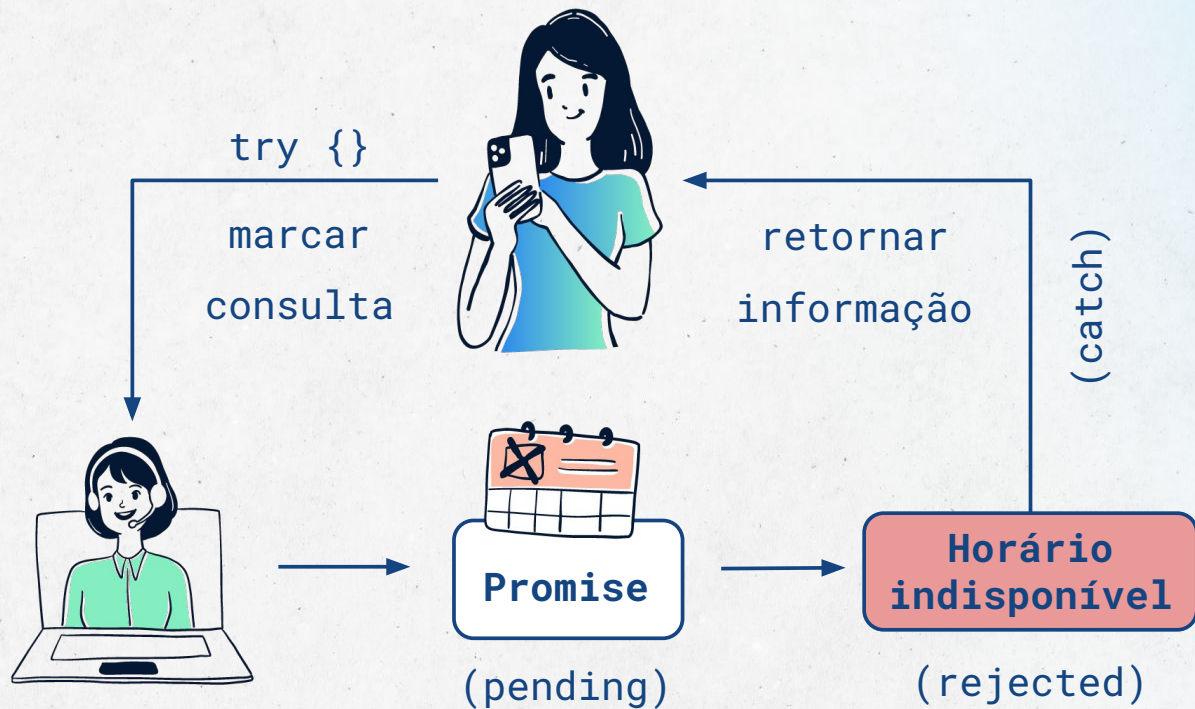
▼ {erro: 'true'} [script.js:5](#)
erro: "true"
▶ [[Prototype]]: Object

>

Esse erro acontece se o CEP estiver errado

JS script.js > ...

```
1  async function buscarEndereco() {
2    try {
3      var consultaCep = await fetch('https://viacep.com.br/ws/35188300/json/');
4      var consultaCepConvertida = await consultaCep.json();
5      if (consultaCepConvertida.erro) {
6        throw new Error('CEP não existente!');
7      }
8      console.log(consultaCepConvertida);
9    } catch (error) {
10     console.log(error);
11   }
12 }
13 buscarEndereco();
```



Mas lembra do problema?

A nossa secretária vai nos passar VÁRIAS opções, e se eu tenho várias opções, significa que eu tenho um ARRAY de dados.

JS script.js >  buscarEndereco

```
1  async function buscarEndereco(cep) {
2    try {
3      var consultaCep = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
4      var consultaCepConvertida = await consultaCep.json();
5      if (consultaCepConvertida.erro) {
6        throw new Error('CEP não existente!');
7      }
8      console.log(consultaCepConvertida);
9      return consultaCepConvertida;
10   } catch (error) {
11     console.log(error);
12   }
13 }
14
15 let ceps = ['35188000', '35185000', '35188002'];
16 let conjuntoCeps = ceps.map(cep => buscarEndereco(cep));
17
18 Promise.all(conjuntoCeps).then(respostas => {
19   console.log(respostas);
20 });
```

▶ {cep: '35188-000', logradouro: '', complemento: '', unidade: '', bairro: '', ...} [script.js:8](#)

▶ {cep: '35185-000', logradouro: '', complemento: '', unidade: '', bairro: '', ...} [script.js:8](#)

Error: CEP não existente! [script.js:11](#)

at buscarEndereco ([script.js:6:13](#))

at async Promise.all ([index 2](#))

▼ (3) [{...}, {...}, undefined] ⓘ [script.js:19](#)

▶ 0: {cep: '35188-000', logradouro: '', complemento: '', unidade: '', bairro: '', ...}

▶ 1: {cep: '35185-000', logradouro: '', complemento: '', unidade: '', bairro: '', ...}

2: undefined

length: 3

▶ [[Prototype]]: Array(0)

>